Predicting Kick Cars at Auto Auctions with Machine Learning

BUAN 6341.001: APPLIED MACHINE LEARNING

SUBMITTED BY - GROUP NO. 9

VENKAT REDDY MALIPATEL,

ASHLESHA SANJAY KADAM,

TANMAY RAJU SHEDGE,

SRI LALITHA MANASWINI GABBITA,

VENKATA SAI CHARAN REDDY BOOJALA,

Table of Contents

Section No.	Section Title	Page No.
1	Motivation	2
2	Data	2
	Data Cleaning	2
	Data Preprocessing	2
	Exploratory Data Analysis	3
3	Model	3
	Logistics Regression	3
	Random Forest	3
	XGBoost	4
	LightGBM	4
4	Model Comparison	5
5	Conclusion	8
	Appendix	9

1. Motivation

In this project, we employ machine learning techniques to distinguish between "good buys" and "bad buys" in vehicle auctions. A "good buy" is likely to yield a profit upon resale, while a "bad buy" could result in losses due to factors such as high repair costs or low resale value. Our models use historical auction data to aid in more informed decision-making, thus enhancing operational efficiency and reducing risk.

2. Data

The dataset comprises vehicle auction data, encompassing aspects like acquisition costs, pricing, and specifications, including:

Numeric Features: Acquisition cost, retail prices, odometer readings, and warranty costs.

Categorical Features: Vehicle make, model, color, and auction location.

Target Variable: IsBadBuy, indicating "good buy" (0) or "bad buy" (1).

Challenges:

Missing Values: Various features exhibit missing data necessitating imputation strategies.

Class Imbalance: A majority of the data labels vehicles as "good buys," skewing the dataset.

Redundant Features: Removal of non-contributory features like ID columns.

	Unnamed: 0	id	IsBadBuy	Auction	VehicleAge	Make	Transmission	VehOdo	Nationality	Size	...	VNST	VehBCost	IsOnlineSale	WarrantyCost	Basic	Mid-level	Luxury	Sport	Special	Trim_Category
0	0	1	0	ADESA	3	MAZDA	AUTO	89046	OTHER ASIAN	MEDIUM	...	FL	7100	0	1113	0	0	0	0	0	Other
1	1	2	0	ADESA	5	DODGE	AUTO	93593	AMERICAN	LARGE TRUCK	...	FL	7600	0	1053	0	0	0	1	0	Sport
2	2	3	0	ADESA	4	DODGE	AUTO	73807	AMERICAN	MEDIUM	...	FL	4900	0	1389	0	1	0	0	0	Mid-level
3	3	4	0	ADESA	5	DODGE	AUTO	65617	AMERICAN	COMPACT	...	FL	4100	0	630	0	1	0	0	0	Mid-level
4	4	5	0	ADESA	4	FORD	MANUAL	69367	AMERICAN	COMPACT	...	FL	4000	0	1020	0	0	0	0	0	Other

5 rows × 31 columns

Figure 1. Sample of the Vehicle Auction Dataset

Data Cleaning & Preprocessing:

Irrelevant Columns Removal: Columns like IDs that do not impact model predictions were dropped.

Data Type Conversions: Transformed numeric-like strings to numeric formats.

Missing Value Handling: Median imputation for numeric features to reduce outlier impact and mode imputation for categorical features.

Feature Transformation: Implemented one-hot encoding for categorical variables and normalized numeric features using StandardScaler.

Class Imbalance Addressing: Applied SMOTE to balance the distribution of the target variable.

Figure 2. Balanced Class Distribution Post-SMOTE Application

Exploratory Data Analysis (EDA)

Target Variable Distribution: Visualized the skewed distribution and addressed it through SMOTE.
Correlation Analysis: Identified strong correlations among price-related features using heatmaps.
Feature Insights: Analyzed significant predictors such as vehicle age and odometer readings.

3. Model

Models Used and Hyperparameter Tuning:

A. **Logistic Regression**:
   Why Used: Logistic Regression serves as a simple and effective baseline for binary classification.
   Hyperparameters Tuned:
   max_iter: Set to 1000 to ensure convergence given the potentially large dataset.
   Regularization strength; inverse of regularization (default grids include values like 0.01, 0.1, 1, 10) to control overfitting.

B. **Random Forest:**

Why Used: Offers robustness against overfitting and can handle large datasets with a mix of numerical and categorical features.

Hyperparameters Tuned:

n_estimators: Number of trees in the forest (tested values: 100, 200, 300).

max_depth: Maximum number of levels in each tree (values like 10, 20, or None for unlimited depth).

C. **XGBoost (Extreme Gradient Boosting):**

Why Used: Known for its speed and performance, particularly on structured data.

How it Works: Boosting algorithm that builds sequential trees, where each tree learns from the errors of the previous ones.

Hyperparameters Tuned:

- **eval_metric: 'logloss' for binary classification.**
- **learning_rate: Step size shrinkage used to prevent overfitting (values like 0.01, 0.1, 0.2).**
- **max_depth: Controls the depth of the tree to prevent model from becoming too complex.**

D. **LightGBM:**

Why Used: Efficient with large data sets, and handles categorical features inherently well.

How it Works: Splits the data leaf-wise as opposed to level-wise, leading to better model adjustments.

Hyperparameters Tuned:

- **num_leaves**: Number of leaves in one tree (values like 31, 63, 127).
- **max_depth**: Limit on tree depth (used -1 for no limit, or specified depths such as 10 or 20)

Rationale for Hyperparameter Choices:

- **Grid Search**: Applied to systematically explore the combination of parameters for each model to optimize performance metrics such as accuracy and ROC AUC.
- **Choice of Parameters**: Focused on controlling model complexity (to avoid overfitting) and improving model learning (speed and accuracy).

Training and Testing

- **Data Splitting**: The resampled dataset (post-SMOTE) was split into training (80%) and testing (20%) sets.
- **Feature Scaling**: StandardScaler was applied to the numeric features before training.
- **Evaluation Metrics**:
- Accuracy
- ROC AUC (Receiver Operating Characteristic Area Under Curve)
- Precision, Recall, and F1-Score
- Confusion Matrix

4. Model Comparison

Logistics Regression -

```
--- Logistic Regression ---
Accuracy: 0.8394
ROC AUC: 0.8959
Classification Report:
              precision    recall  f1-score   support

           0       0.77      0.98      0.86     12933
           1       0.97      0.70      0.81     12670

    accuracy                           0.84     25603
   macro avg       0.87      0.84      0.84     25603
weighted avg       0.87      0.84      0.84     25603

Confusion Matrix:
```

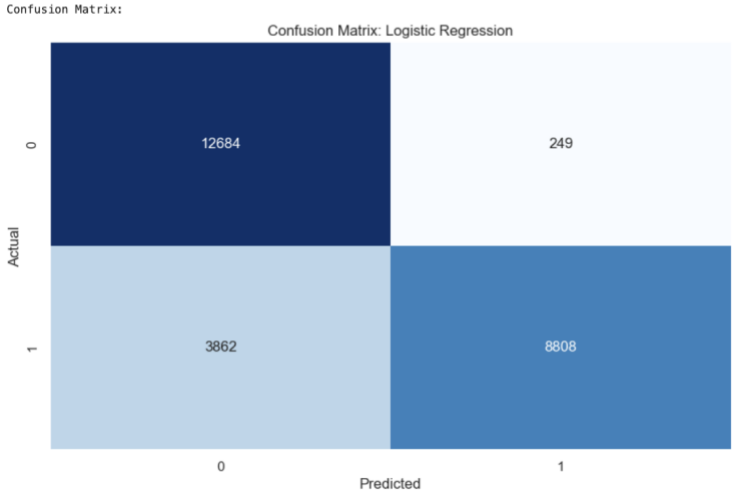

Figure 3. Performance Metrics for Logistic Regression Model

Random Forest -

```
--- Random Forest ---
Accuracy: 0.9312
ROC AUC: 0.9634
Classification Report:
              precision    recall  f1-score   support

           0       0.89      0.99      0.94     12933
           1       0.99      0.87      0.93     12670

    accuracy                           0.93     25603
   macro avg       0.94      0.93      0.93     25603
weighted avg       0.94      0.93      0.93     25603

Confusion Matrix:
```

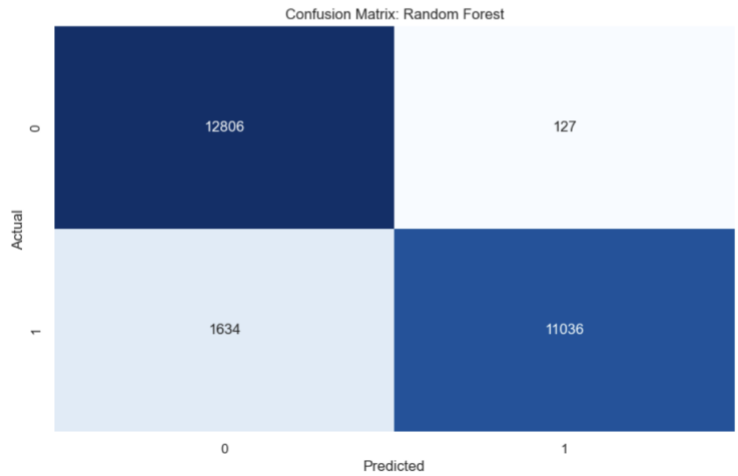

Figure 4. Performance Metrics for Random Forest Model

XGBoost –

```
Accuracy: 0.9216
ROC AUC: 0.9579
Classification Report:
              precision    recall  f1-score   support

           0       0.87      0.99      0.93     12933
           1       0.99      0.85      0.91     12670

    accuracy                           0.92     25603
   macro avg       0.93      0.92      0.92     25603
weighted avg       0.93      0.92      0.92     25603

Confusion Matrix:
```

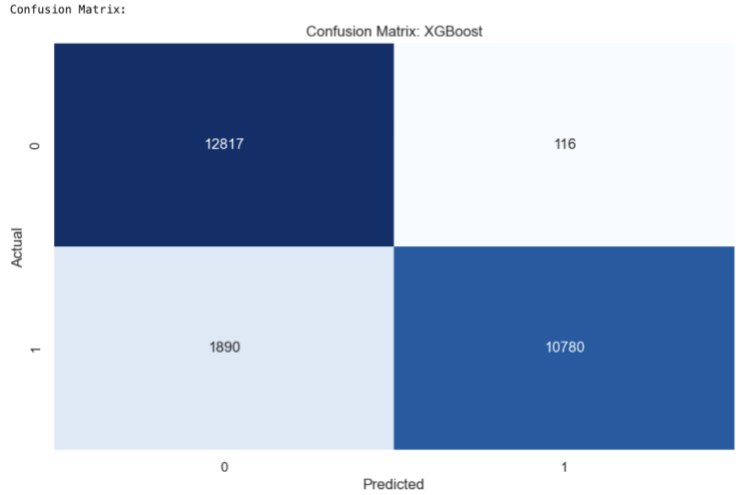

*Figure 5. Performance Metrics for XGBoost Model**LightGBM –***

```
--- LightGBM ---
[LightGBM] [Info] Number of positive: 51337, number of negative: 51074
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.043442 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 20632
[LightGBM] [Info] Number of data points in the train set: 102411, number of used features: 111
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.501284 -> initscore=0.005136
[LightGBM] [Info] Start training from score 0.005136
Accuracy: 0.9194
ROC AUC: 0.9559
Classification Report:
              precision    recall  f1-score   support

           0       0.86      1.00      0.93     12933
           1       1.00      0.84      0.91     12670

    accuracy                           0.92     25603
   macro avg       0.93      0.92      0.92     25603
weighted avg       0.93      0.92      0.92     25603

Confusion Matrix:
```

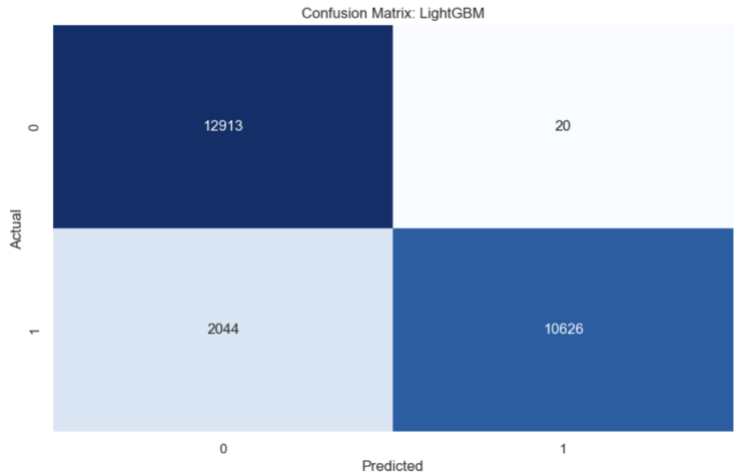

Figure 6. Performance Metrics for LightGBM Model

- **Performance Metrics**: Utilized accuracy, ROC AUC, precision, recall, and F1-score.

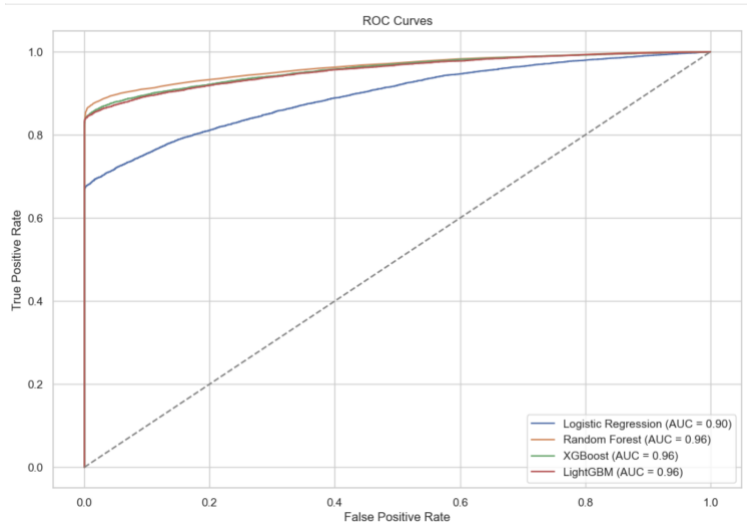

Figure 7. Receiver Operating Characteristic (ROC) Curves for Predictive Models

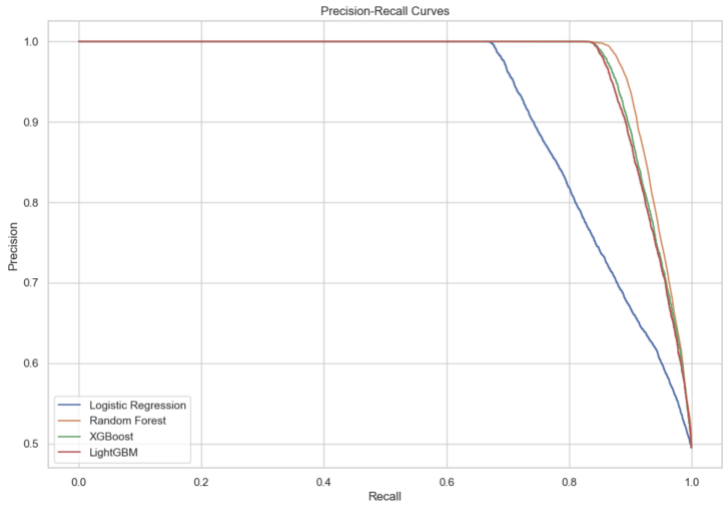

Figure 8. Precision-Recall Curves for Machine Learning Models

The metrics of accuracy, ROC AUC, precision, recall, and F1-score were chosen for model comparison to provide a comprehensive evaluation of each model's performance, considering overall accuracy, ability to discriminate between classes, & balance between precision & recall, crucial for handling class imbalances.

5. Conclusion

Best Model: Random Forest emerged as most balanced & interpretable model, achieving an AUC of 0.963.

Feature Insights: Vehicle age, odometer readings, and acquisition price were critical predictors.

Impact of Preprocessing:

- SMOTE significantly improved recall for the minority class.
- Scaling ensured consistent model performance across features

Appendix

Additional visualizations:

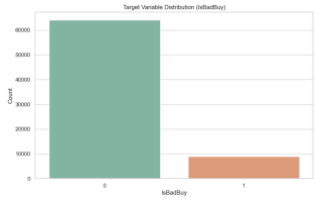

Figure 9. Original Class Distribution of the Target Variable 'IsBadBuy'

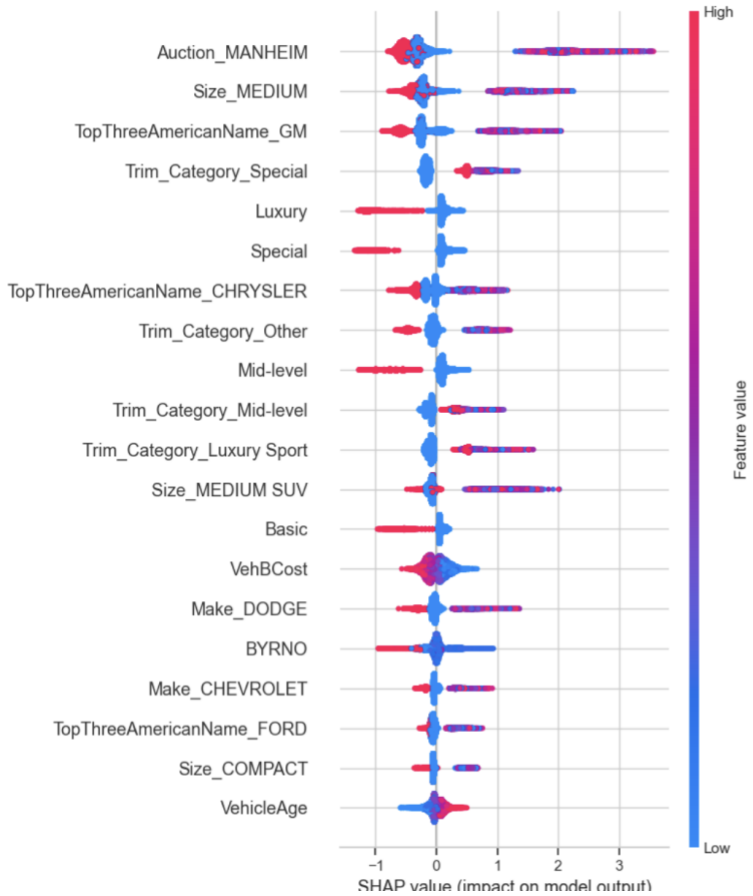

Figure 10. SHAP Value Summary Plot for Predictive Model

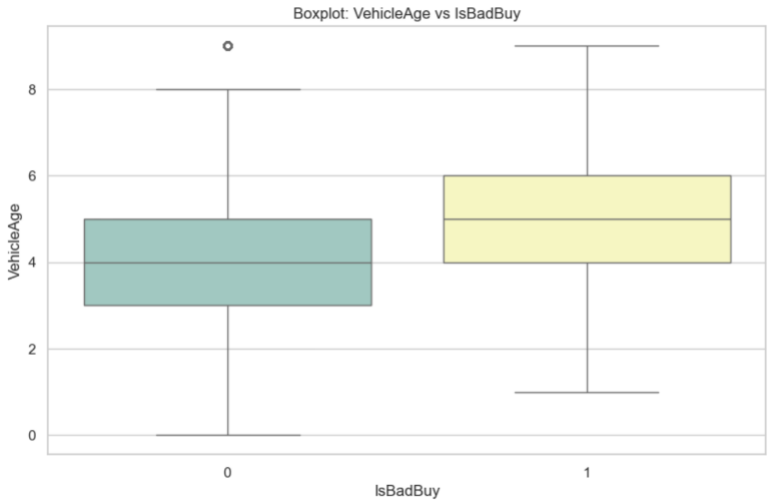

Figure 11. Boxplot of Vehicle Age by IsBadBuy Classification

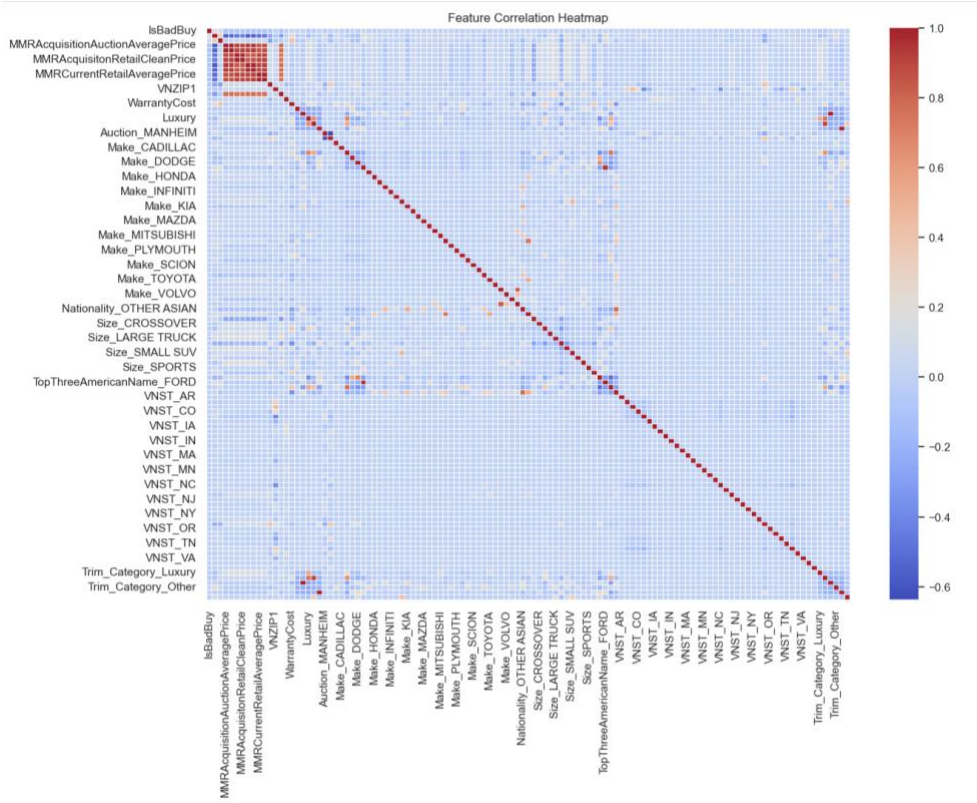

Figure 12. Feature Correlation Heatmap in Vehicle Auction Data

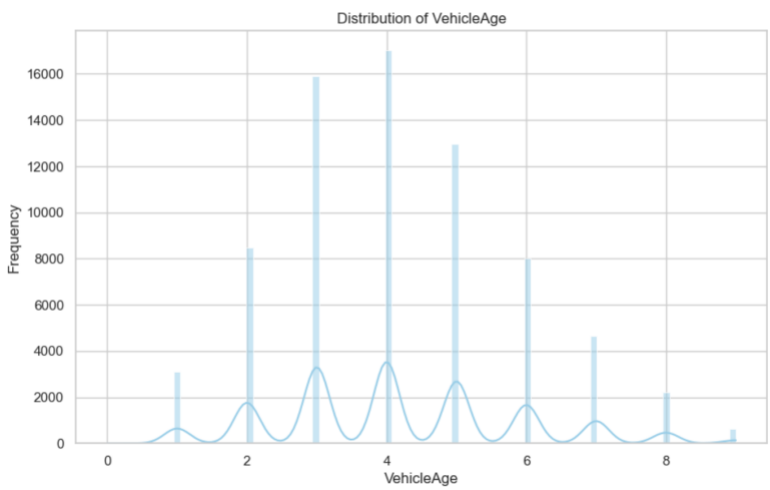

Figure 13. Histogram of Vehicle Age Distribution

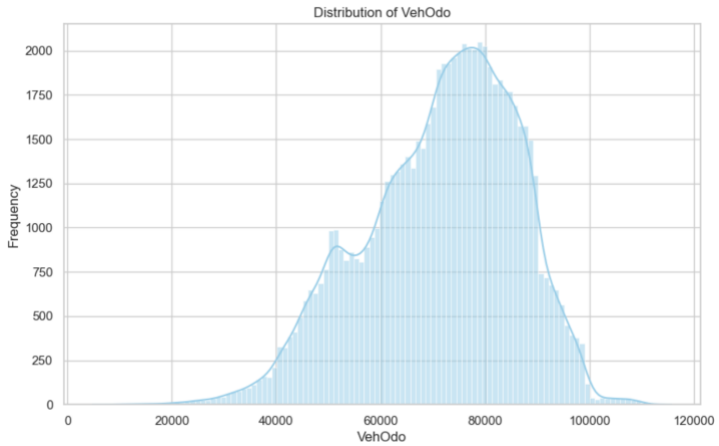

Figure 14. Histogram of Vehicle Odometer Readings Distribution

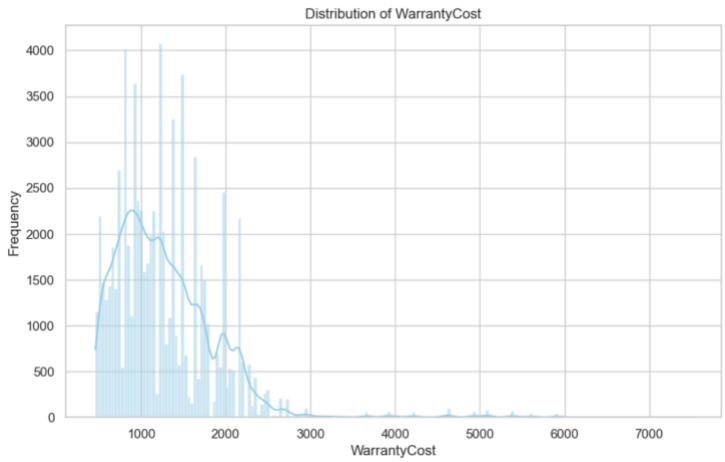

Figure 15. Histogram of Warranty Cost Distribution

References

• Auto Auctions in the U.S. https://www.manheim.com/ and https://www.adesa.com/

• SHAP Analysis https://shap.readthedocs.io/en/latest/

• Oversampling https://imbalanced-learn.org/stable/over_sampling.html#smote

• Gradient boosting https://www.geeksforgeeks.org/ml-gradient-boosting/ and https://www.analyticsvidhya.com/blog/2020/02/4-boosting-algorithms-machine-learning/

• Géron, A. (2017). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly Media.